Controlling and Automating a Model Railway with MersAG_Universal, OpenLCB and JMRI

This User Manual is presented in 4 parts each of which is available as a separate PDF download.

***Important Note** :*
If you're reading this document directly from the Github repository you may find formatting errors. Please open the PDF in your MersAG_Universal PDF Documentation Folder.

Part 1 gives a summary of the technologies we use and why, as model railway enthusiasts, we've chosen to use them. More importantly, it sets out why we believe that it gives the railway modelling community a fairly simple, low cost path to achieving high levels of reliable automation covering "realistic" turnout control and frog switching, signalling, sensing, block detection, lighting, animations and more.
Part 2 is aimed at modellers with little knowledge of electronics and/or, maybe, the fear of soldering!! We walk you through making a very simple, cheap "breadboard" to simulate a red/ green signal from a simple push button switch. We then learn how easy it is, with our MersAG_Universal program, to integrate this with JMRI. Finally, and with just 2 more components and very little effort, we add slow-motion servo motor control. Our aim is to let you decide, with very little time and cost, if our way looks right for you.
In **Part 3** we'll add a few more components to build a "node" that can control multiple turnouts, signals etc and detect multiple sensors for you to control and automate all or part of your layout.
Finally, in **Part 4**, we'll add just 1 more component to enable several nodes to communicate over a CANBUS network that will enable you to control even very large complex layouts reliably.

Part 2 : From Humble Beginnings ....

2.1 What You Need 1

2.2 Assemble and Test the Breadboard 1

2.3 Upload Simple Test Sketch to Your ESP32 2

2.4 Installing MersAG_Universal onto your ESP32 3

2.5 Configure JMRI to "talk" to your ESP32 4

2.6 Configure ESP32 using JMRI 5

2.7 Adding Servo Control Servo 10

Reference 14

To get started on our journey to a fully automated model railway we don't need to buy any expensive gear, we don't need to spend a lot of time making up complex circuit boards, we don't need to be an electronics master or programmer : actually, we don't even need to solder!!
In this section we'll put together a very basic breadboard and test it. We'll then link it to JMRI and see how easily you can set JMRI up to control pretty much all of your layout using an ESP32 with our Universal sketch and JMRI.
If you're already experienced in making up circuits, please feel free to motor on to 2.4, this section is aimed at less experienced modellers.

2.1 What You Need

- Breadboard (not essential but makes the assembly much easier)
- 30 Pin ESP32 with or without a 30 PinShield
- A push button switch
- 2 x LEDs (Red and Green would be good but any colour will do)
- 2 x 220Ω resistors
- A few Dupont connectors (ideally of a few different colours)
- A suitable computer such as a Windows 10 or 11, MacBook or Linux Mint 20

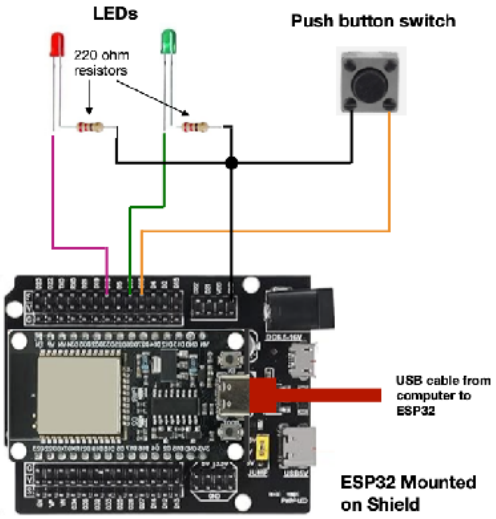


Fig 2.1 Test Breadboard Schematic

2.2 Assemble and Test the Breadboard

We're going to assemble the board with a push button and 2 LEDs according to the schematic (Fig 2.1).

Always make sure that the ESP32 is not connected to your Computer when making connections to it or its Shield.

Firstly, place the push button and 2 LEDs on the breadboard, something like as shown in Fig. 2.2.

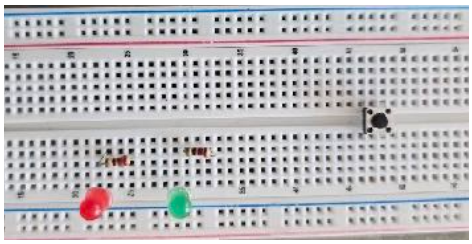


Figure 2.2 Place LEDs and Push Button on the breadboard

Note : The long legs of the LEDs are to the left.

Note : Words in green and underlined are links to the Reference section that gives a bit more technical explanation.

Then place the 2 resistors to connect to the right (short) legs of the LEDs. You can test the LEDs by connecting the left leg to the +'ve side and the resistor from the right leg of the LED to the -'ve side of a 3v battery. If the LED doesn't light, switch the 2 sides.

Finally, make the following connections with Dupont wires :

- Right leg of both LEDs to their respective resistors
- Right side of both resistors to GND
- GND to GND on ESP32 (use 1 of the outer 4 GND pins in the upper right of the ESP32)
- One side of the push button to GND.
- Other side of push button to D16 (we're wiring the push button as a pull-up switch in this example)
- Left side of green LED to D17
- Left side of red LED to D18

***Note** : ESP32 pins are not in numeric order.*

Hopefully your breadboard will look something like that

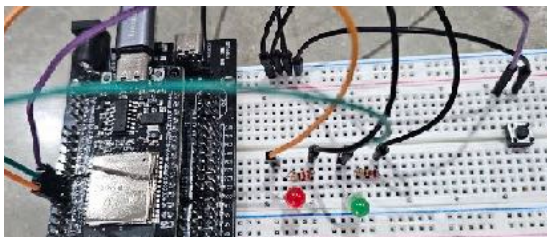


Figure 2.3 Your breadboard should look something like this.

shown in Fig 2.3. Before connecting to your computer, double check that the connections are all good and correct - we don't want you to fry your ESP32 due to any electrical shorts!
When you're ready, connect your computer to the ESP32 via a USB cable and check that the LED on the ESP32 is lit.

2.3 Upload Simple Test Sketch to Your ESP32

You now need to upload a simple **sketch** to your ESP32 to test that your breadboard works as planned. To do this :

- Connect your ESP32 to your computer using a **USB data cable**, not a recharging cable as this may cause uploading faults. Make sure you connect to the 'C' socket on the ESP32, not the shield.
- Open your Arduino IDE and select File > New Sketch

```
// Simple sketch to toggle 2 LEDs each time push-button switch is pressed
#define BUTTON_PIN 16 // Button
#define LED_PIN1 17 // Green LED
#define LED_PIN2 18 // Red LED
int led_state = LOW;
int button_state, last_button_state;
void setup() {
pinMode(BUTTON_PIN, INPUT_PULLUP); pinMode(LED_PIN1, OUTPUT); pinMode(LED_PIN2, OUTPUT);
last_button_state = digitalRead(BUTTON_PIN); Serial.begin(115200);
delay(1000);
}
void loop() {
button_state = digitalRead(BUTTON_PIN);
if (last_button_state == HIGH && button_state == LOW) {
led_state = !led_state; digitalWrite(LED_PIN1, led_state); digitalWrite(LED_PIN2, !led_state);
Serial.println(led_state);
}
last_button_state = button_state; delay(500);
}
```

- Copy the following sketch and paste it into your Arduino IDE, replacing any default text

Make sure that you select the correct :

- Board : **ESP32 Dev Module**
- USB Port (in the **Tools** dropdown menu)
- Upload Speed of **115,200** (further down in the same menu)

You can then go ahead and Compile and Upload your sketch. Once uploaded you should find that :

- On pressing your push button, the Green LED turns on.
- On a second press, the Green LED turns off and the Red turns on.
- On subsequent presses, the 2 LEDs will toggle between these 2 states.

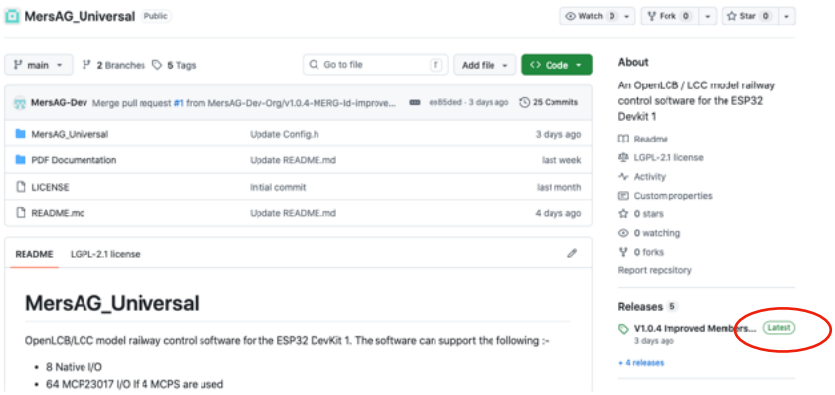
Wow, you may say, that's OK, but so what?! Well, first we've shown that we can toggle signal lights at the touch of a button, like on a mimic panel. More importantly, we're now ready to connect this to JMRI to see how we can get JMRI to recognise a button press and do a countless number of things to automate our layout : controlling servos, frogs, signals, level crossings, lights in buildings, on roads and stations and more. We can also detect more than "just" push buttons

such as block detectors, infra red position sensors etc. And we can get JMRI to take action when something is detected!
But surely, you may ask, to do all that requires a hell of a computer program to sit between JMRI and my layout? And you'd be right of course : So let's meet "MersAG_Universal".

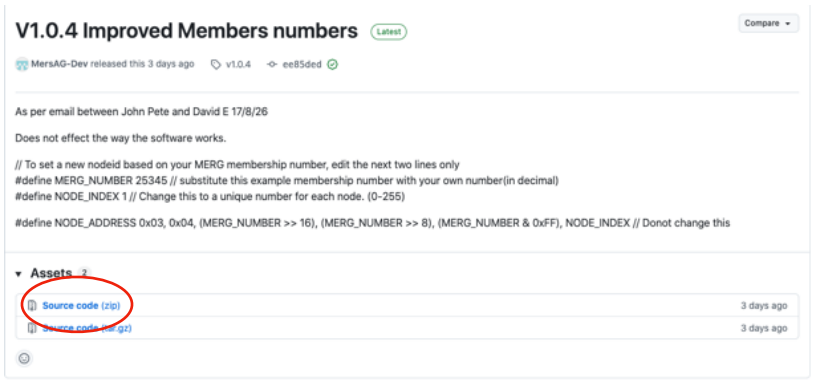
2.4 Installing MersAG_Universal onto your ESP32

MersAG_Universal is a Sketch created to enable numerous devices such as servos, sensors, block detectors, LEDs etc to automate a broad range of model railway accessories.
To continue on our step-by-step approach, we'll just start by controlling the simple breadboard you assembled in 2.2. Firstly we need to download MersAG_Universal and instal it on our ESP32 :

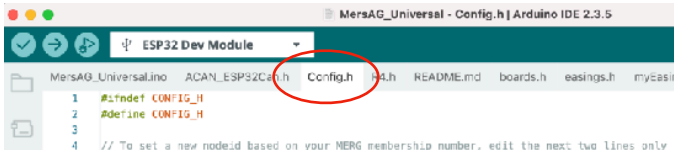
1. Click the link : **https://github.com/MersAG-Dev-Org/MersAG_Universal** to open the repository housing the MersAG_Universal sketch
2. At the bottom right of the opening screen press **Latest**



3. At the bottom left of the next screen, press Source Code (zip) to download the file. Once the zip file has downloaded, click on it to expand it to a folder and save the folder as required.
4. Prior to compiling the sketch the following third part libraries are required to be installed using the Arduino Library manager.



   - OpenLCB_Single_Thread by David Harris, Version 0.1.19
   - mcp23017 by Bertrand Lemasle tested with version 2.0.0
   - ESP32Servo by Kevin Harrigton tested with version 3.2.1
   - ServoEasing by Armin Joachimsmeyer tested with version 3.6.0
   - ACAN_ESP32 by Mohamed Irfanulla tested on version 3.0.3
5. Click on the MersAG_Universal.ino file to open it.
6. When it's open, select the Config.h tab at the top of the sketch.



7. You MUST force a reset of your ESP32's EEPROM to factory Defaults. To do this, simply replace the "0" at the end of the line "#define RESET_TO_FACTORY DEFAULTS " with a "1".

```
// To Force Reset EEPROM to Factory Defaults
// Need to do this at least once.
#define RESET_TO_FACTORY_DEFAULTS 1
```

8. Before compiling and uploading your modified MersAG_Universal Sketch, it's worth checking your connection settings : From the Tools menu, make sure that your :

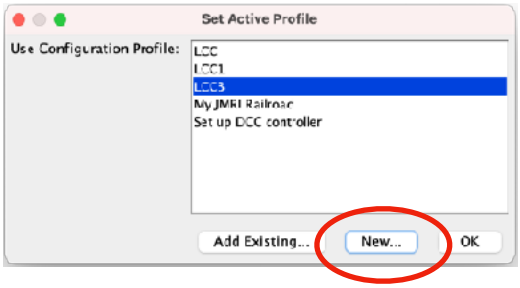
   - Board is set to ESP32 Dev Module,
   - Port is set correctly, and
   - Upload Speed is "115200".

   Make changes if necessary and then compile and upload your sketch to your ESP32.

> ***Note :*** *When compiling and uploading you'll see a stream of message in red that can be mistaken for error messages. Please ignore them as they're not error messages.*

9. After uploading is complete **you must go back to your Config.h** screen and change the value of RESET_TO_FACTORY_SETTINGS **back to O** and upload your sketch again.

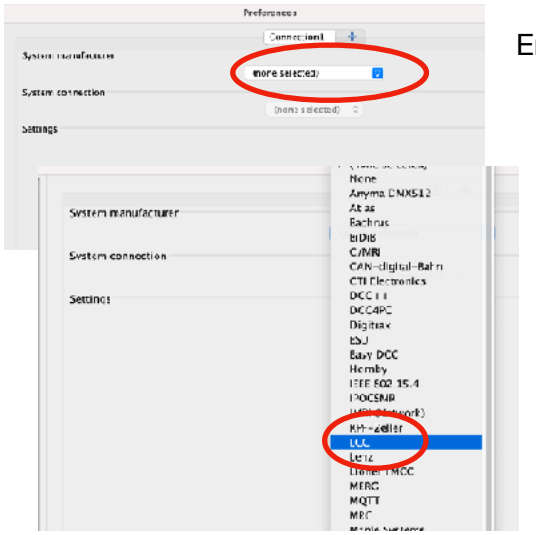
Once uploaded you're ready to configure JMRI.



2.5 Configure JMRI to "talk" to your ESP32

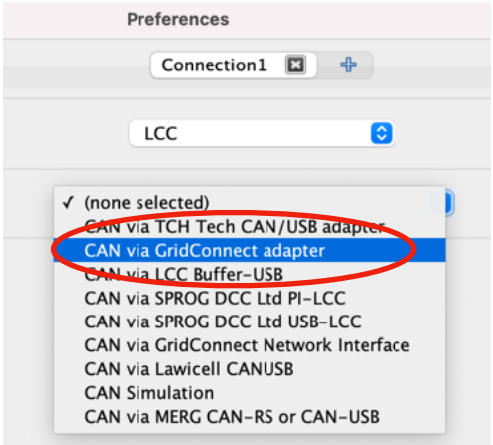
With your Breadboard still connected, Open **PanelPro**.
In the Set Active Profile window, select **New**.

Enter your Profile name and click OK.

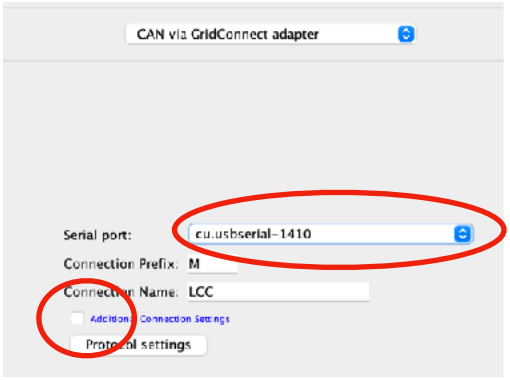


> ***Note*** *: Screenshots were captured on a MacBook Pro running Big Sur ver 11.7.11 and JMRI ver 5.16. You may see differences in screen layout on your system.*
> *You may also need to select Edit —Preferences after entering your profile name.*

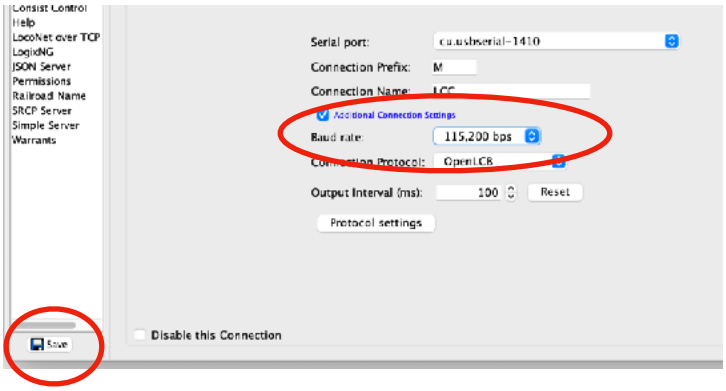
Select the System Manufacturer by opening the drop down window, cursoring down and selecting **LCC**.



Select System Connection by opening the drop down window, cursoring down and selecting **CAN via GridConect adapter**



Select your Serial (COM) Port from the dropdown box and then click on **Additional Connection Settings**



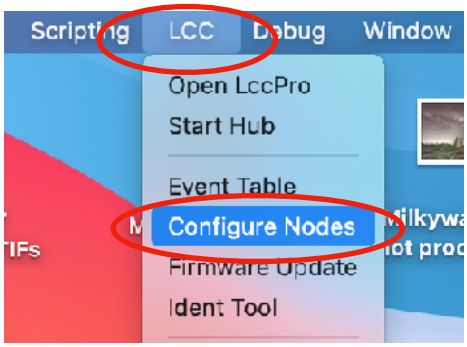
Finally, select **115,200** bps as your Baud Rate and don't forget to save your configuration by pressing the **Save** button tucked away in the lower left corner.

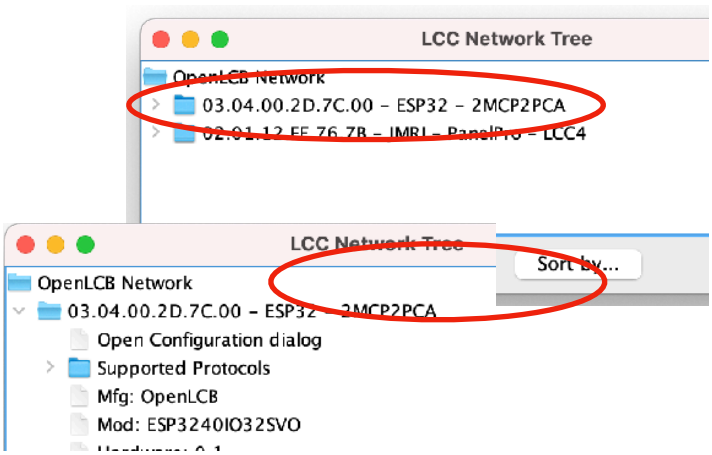
So, JMRI is now configured to your ESP32 sketch and we're ready to use JMRI to configure MersAG_Universal

2.6 Configure ESP32 using JMRI

So, we've set up our test Breadboard and proved that it works, we've uploaded our Universal sketch to our ESP32 and we've set JMRI up to interact with that sketch.
Now we want to get JMRI to "teach" our ESP32 to "listen" for our button press and "teach" it what to do with our LEDs whenever it is pressed.

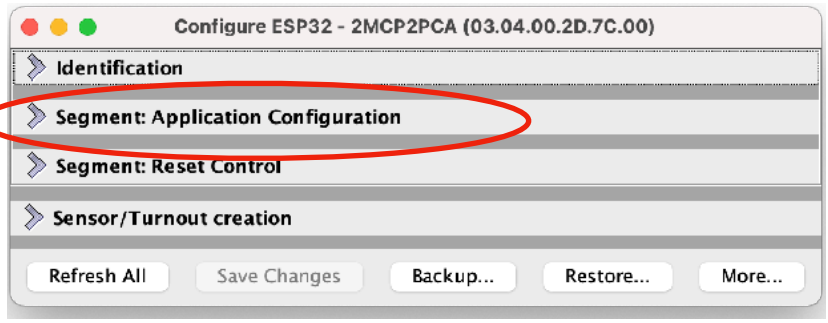
You'll see that you now have a new tab in the top horizontal menu bar of our PanelPro Window - **LCC**. Click on this tab and select **Configure Nodes** in the drop-down menu.



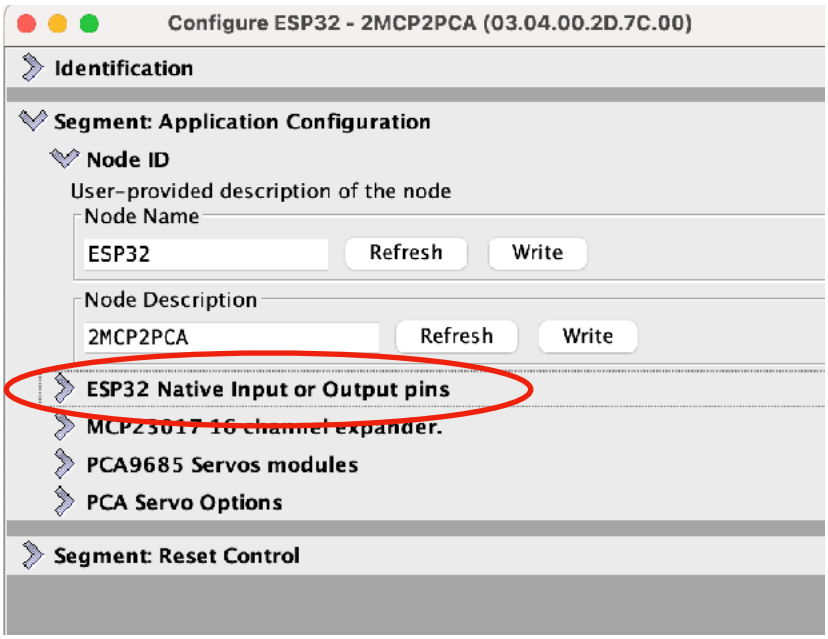


Another window opens up called **LCC Network Tree**. Expand the **ESP32 - 2MCP2PCA** panel and select **Open Configuration dialog** to open the **Configure ESP32 - 2MCP2PCA** window.

In the **Configure ESP32 - 2MCP2PCA** window, expand the **Segment: Application Configuration** window. This window may take a few seconds to open as it needs to gather data that may already have been saved.

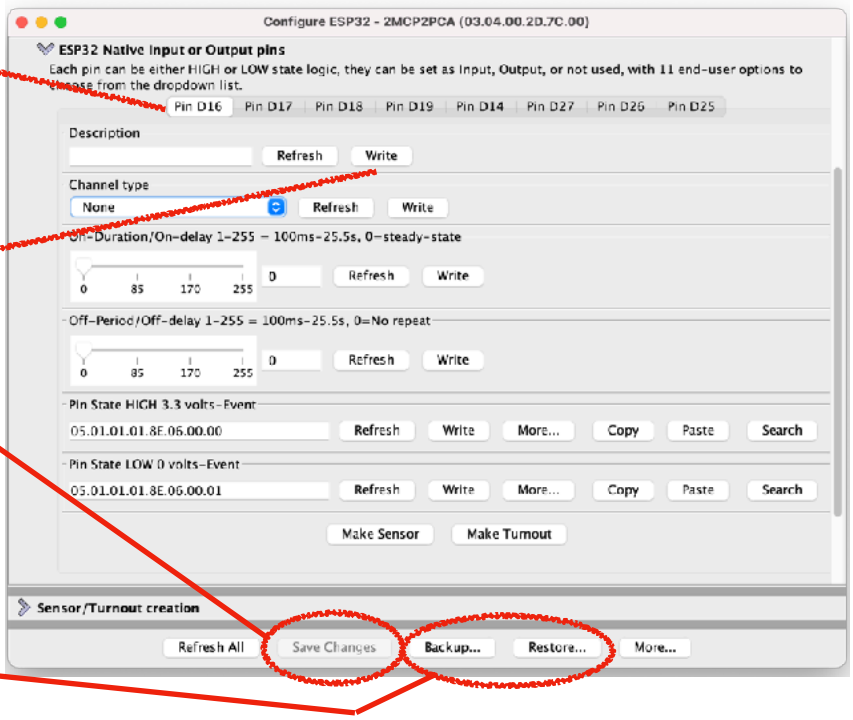


The expanded window allows a number of options which can also then be expanded. For now, expand the **ESP32 Native Input or Output pins** so that we can configure the pins connected to the push button and LEDs on our breadboard.



Let's take a look at some of the key features in the **ESP32 Native Input and Output pins** window :

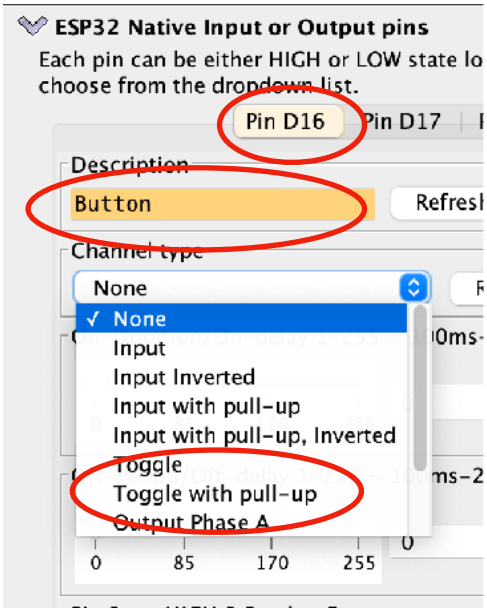
- Each of the available I/O pins can be selected and assigned a number of parameters, some of which we'll be assigning next.
- You can **Write** additions/ edits to memory each time you make a change.
- **Or** you can **Save Changes** you've made.
- As you add more complexity to your configuration we recommend that you make frequent **Backups** that you can **Restore** as and when required.



Before we start configuring pins it's a good idea to Backup our configuration in JMRI and then make regular Backups throughout the configuration process. That way, if things don't work as expected for any reason you can go back to an earlier "working" configuration rather than editing or deleting changes made subsequently.

Let's now configure the 3 pins connected to the ESP32 on our breadboard : **Pin 16** to the push button, **Pin 17** to the green LED and **Pin 18** to the red LED.

Select **Pin D16** and, in the Description box, label this as e.g. "Button".
Notice that both of these windows are now yellow : that's to tell us that they've been changed but the changes have not been saved.

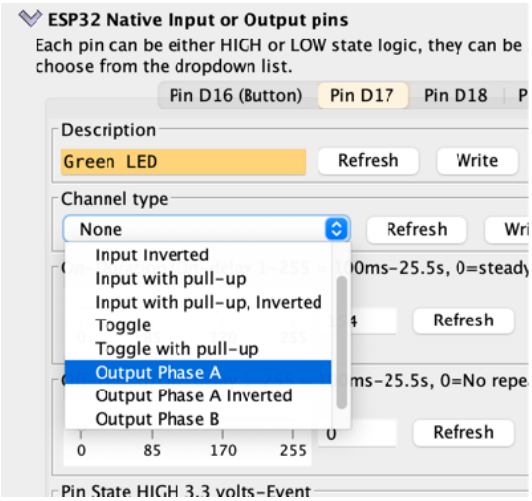


Let's also expand the **Channel type** drop-down window and select "**Toggle with pull-up**". That's because :

- we want our LEDs to toggle between on and off each time the button is pressed, and
- we've wired our push button as a **pull-up** to the 3.3v supply from our ESP32 each time we press it.

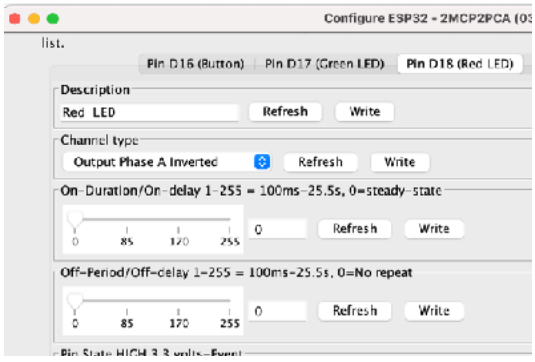
Make sure that you press "**Save Changes**" so the Pin 16 and Description windows are no longer yellow, and the Description and Channel type windows have been saved.

Let's now configure our LED pins. Select **Pin D17**, enter "**Green LED**" as the Description and select **Output Phase A** as the **Channel type**.



Again press "**Save Changes**". If you press your push-button on your breadboard now, the Green LED should turn On and then toggle Off and On with each subsequent press. (It's best to only press every second or so to allow the LED time to react).

Let's now select **Pin D18** and enter "Red LED" as the Description. But this time, we want the Red LED to turn off when the Green LED is on and *vice versa* so we'll select "**Output Phase A Inverted** as the Channel type. Again, remember to press Save.

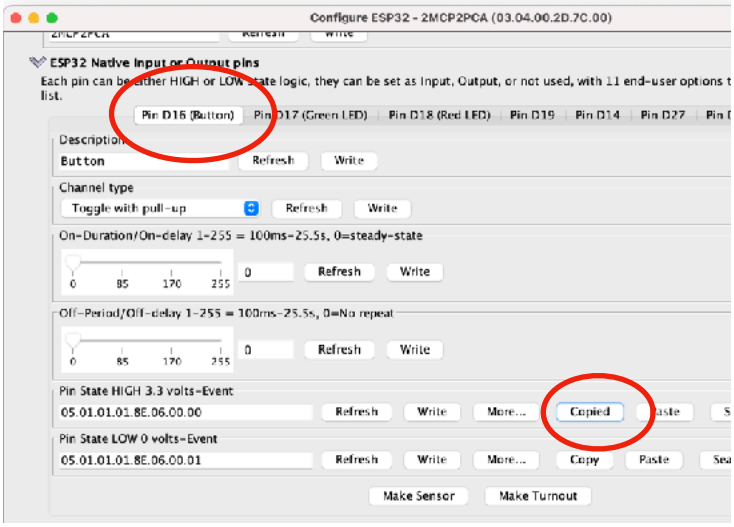


Now that you've configured your 3 pins and saved the configurations it would be a good time to **Backup** your work by pressing the Backup button at the bottom of the ESP32 Native Input or Output pins window. That

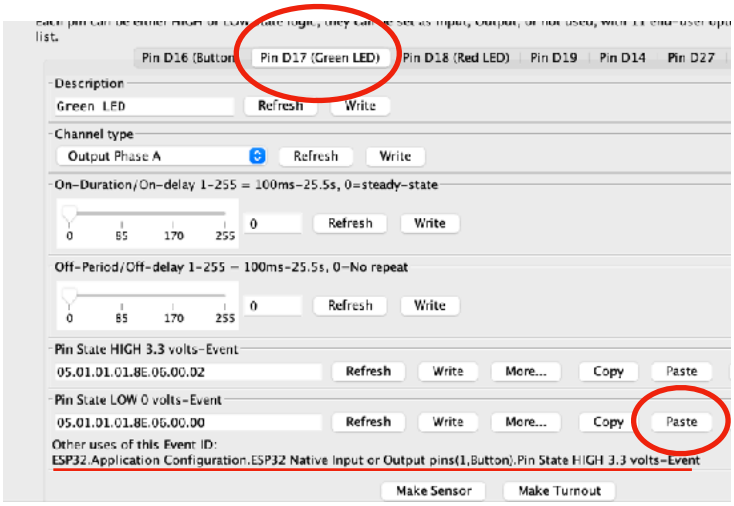
way, if you make any errors in the next operation(s) you can always Restore to this point in our process. So don't be afraid to try a few things out!

Our last step in his process is to "Teach" our LEDs to turn on and off as we intend.

Select **Pin D16** (Button) and press the **Copy** button for the **Pin State HIGH 3.3 volts event**. The button will then change to **Copied**, confirming that this Event has been copied to the clipboard.



Next, Select the **Pin D17** (Green LED) and press the **Paste** button for the **Pin State HIGH 3.3 volts event.** Note that the event created for Pin 16 being pulled HIGH has been copied to the pin state LOW event for the Green LED which will make it turn OFF.
Note that the event has been copied into the Pin State LOW 0 volts-Event window.

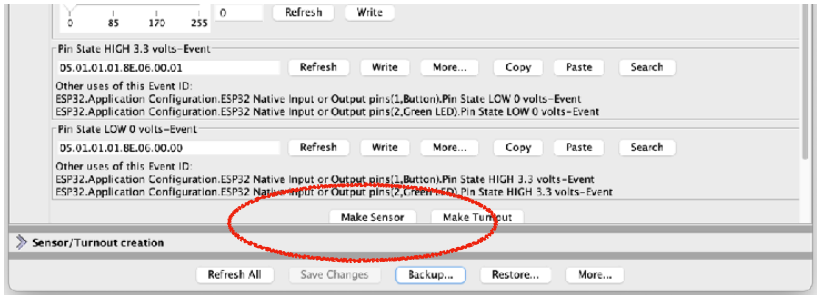


Again Select Pin D16 (Button) but, this time, press the **Paste** button for the **Pin State LOW 0 volts event.** Copy this event pin state HIGH event for the Green LED.
Note that this event has been copied into the **Pin State HIGH 3.3 volts-Event** window.

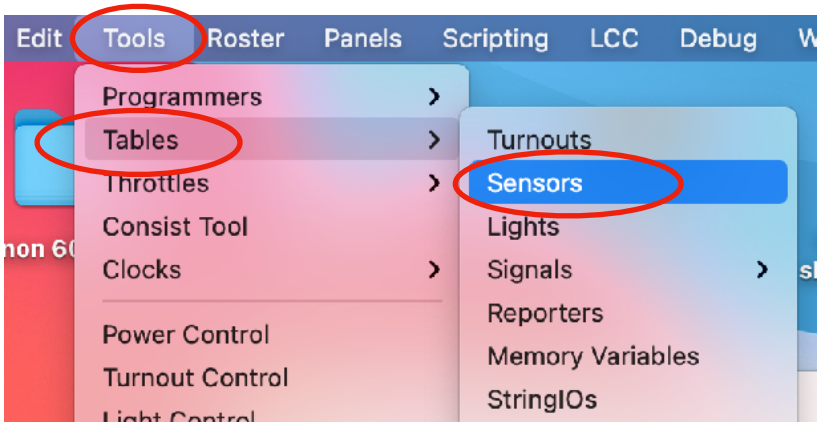
Finally, repeat the above steps to copy both of the Pin 16 HIGH and LOW events into the LOW and HIGH events for Pin 18 (Red LED). You should now see that each time you press the push button your breadboard, the Red and Green LEDs should toggle On-Off and Off-On respectively.

Using this example you can see how easy it would be to get LED signals working on your layout. You'll need more pins available if you want more than a few Signals but we'll see how to add lots more in Part 3. But, before we take the next step, let's back up our work and then get our signals working from a virtual switch in PanelPro. Having an on-screen layout of your track with a "switch" on each turnout gives you the computer equivalent of a mimic panel, but without all the wiring.

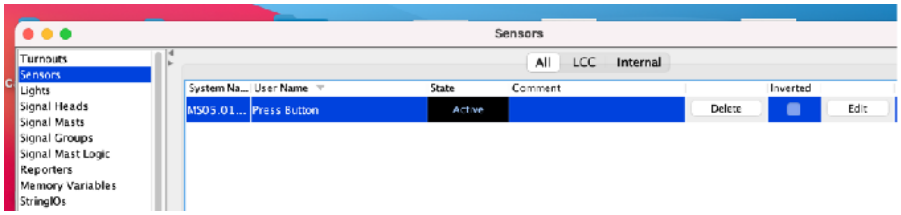
While we still have our **ESP32 Native Input or Output** window open, select Pin 16(Button) and then just click on the Make Sensor button at the bottom of the window. This makes a virtual button for you.



If you now minimise the **Configure ESP32** window you've been working in and click on the **PanelPro** window you can then select **Tools - Tables - Sensors** from the top menu bar.



When you select Sensors, a Sensors window opens and there is your virtual switch. If you press it, the State will toggle between Active and Inactive and your LEDs will toggle. And if you press the push button on your breadboard you'll also see the State change here as well.



So, you've seen how you can easily put a breadboard together to demonstrate toggling two LEDs with a push button switch to simulate a signal on your layout. Having tested the board you've uploaded our MersAG_Universal open-source sketch onto your ESP32 that you've then connected with JMRI. In JMRI you've set the press button switch up as a sensor that initiates actions - toggling the signal - whenever either the push button is pressed or the 'virtual' button you've created in JMRI is pressed.

But, probably the main thing you want to see is how you can make your points/turnouts work. Even to make them work nice and slowly to bring a sense of realism to the way they operate. Well, for just a few £/$/€ or whatever you can experience one of the many major benefits of MersAG_Universal!

All you need is a small extension board called a PCA9685, a servo and a few more Dupont wires - all available from Amazon, Ali-Express and other outlets.





Wire the PCA to your breadboard according to the schematic in Fig 2.4.

Fit Dupont cables from your ESP32 to your PCA9685 as follows :

- preferably Black, from GND on the to GND
- preferably Red, from VCC on the to VCC
- preferably Green, from Pin D21 to SDA
- preferably Yellow, from Pin D22 to SCL

These 4 wires form an **I2C Data Bus** that enables several PCA9685s and other expansion boards, each with their own unique address, to be “daisy-chained” together. This enables very large numbers of I/O pins to be available to the ESP32 and dozens of servos as we’ll see in Part 3!

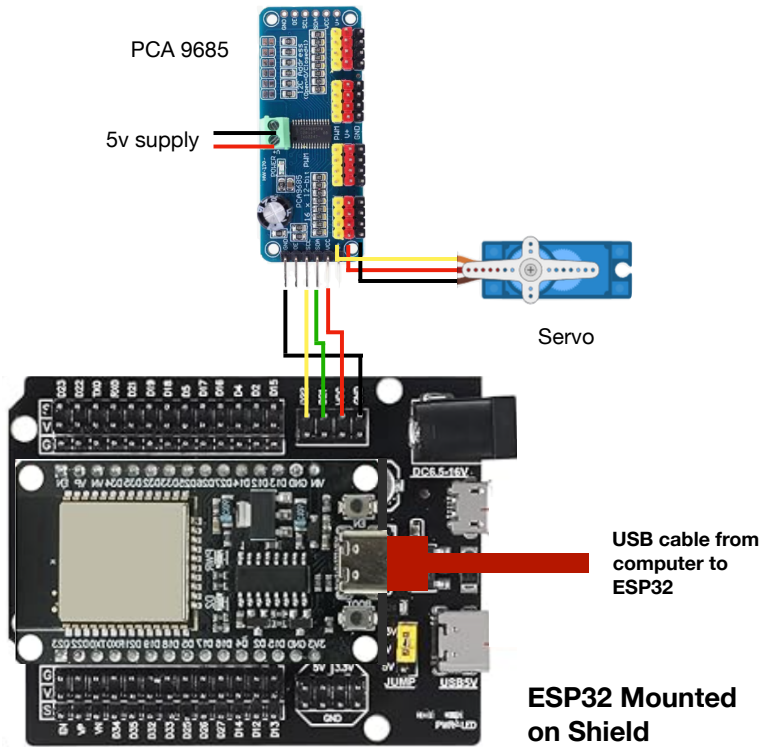


Fig 2.4 Schematic showing additional wiring to breadboard

Connect your Servo to the first row of pins on your PCA9685.

Finally, connect a 5v power supply to your PCA9685. Note, the 3.3v supply on the ESP32 may not be strong enough to power the Servo so it’s best to use an external power source.

When all of your additional connections are made on your breadboard, ESP32, PCA9685 and Servo should look something like that shown in Fig 2.5.

Our MersAG_Universal sketch has all of the operating instructions to control our servos so we can now add our servo in JMRI.

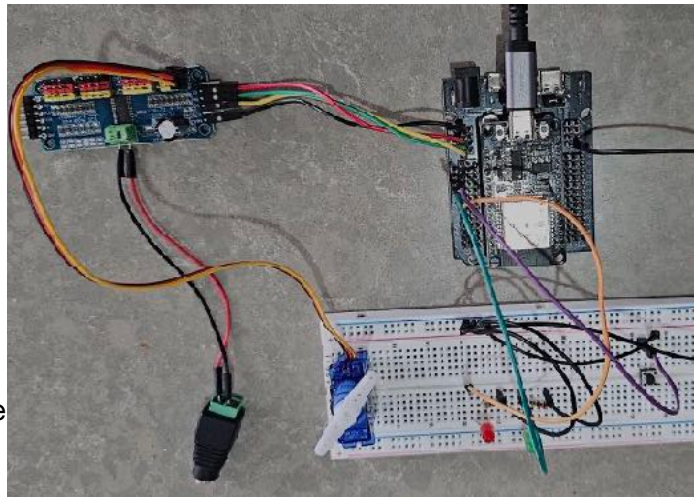
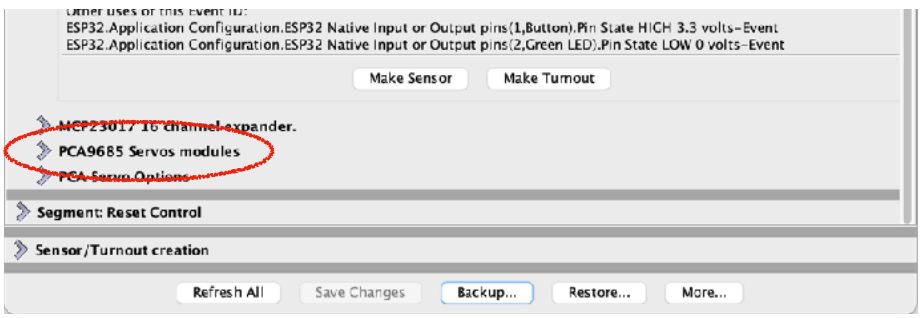


Fig 2.5 Breadboard, ESP32, PCA9685 and Servo all connected up

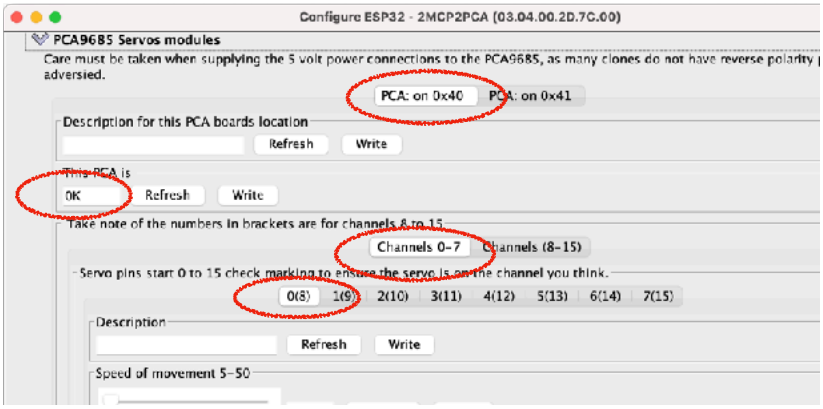
2.7 Adding Servo Control Servo

Scroll down below the end of ESP32 Native Input and Output Pins section and expand the **PCA9685 Servo modules** section.



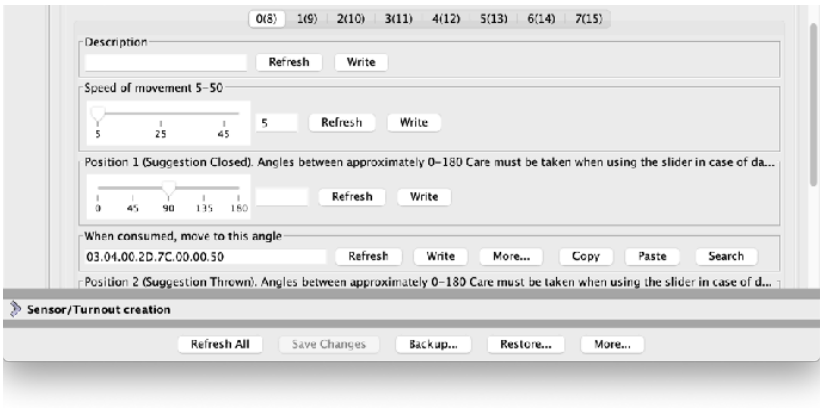
The upper part of the expanded PCA9685 Servo modules window shows that :

- JMRI has detected a PCA9685 with the address 0x40 (this is the default address of all new PCA9685s)
- the PCA8685 connection is OK
- Channel 0 is active : that's because you've connected your servo to the first set of servo pins.



If you scroll down, for this servo you can enter :

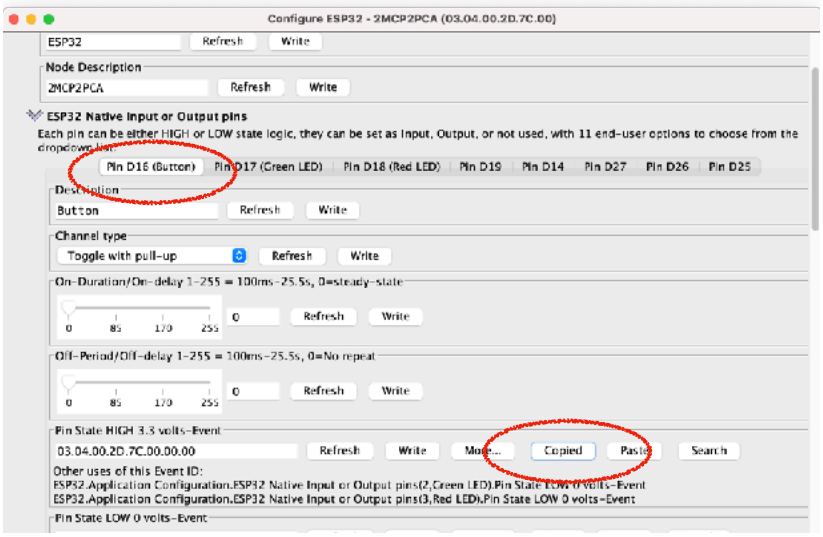
- Description (perhaps the position of the servo/point on your layout)
- Set the speed of movement
- Set the angles (in degrees) for the two positions (Closed and Thrown)



For this test, set the speed to **25**, Position 1 to **135** and Position 2 to **45**.

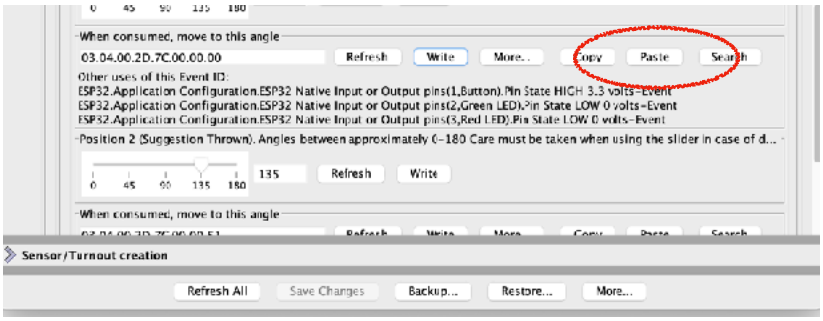
As we did earlier to connect the LEDs to the Push Button event on our breadboard, we can now also connect the servo.

Scroll back up to the ESP32 Native Input and Output Pins section and select Pin D16 (Button).



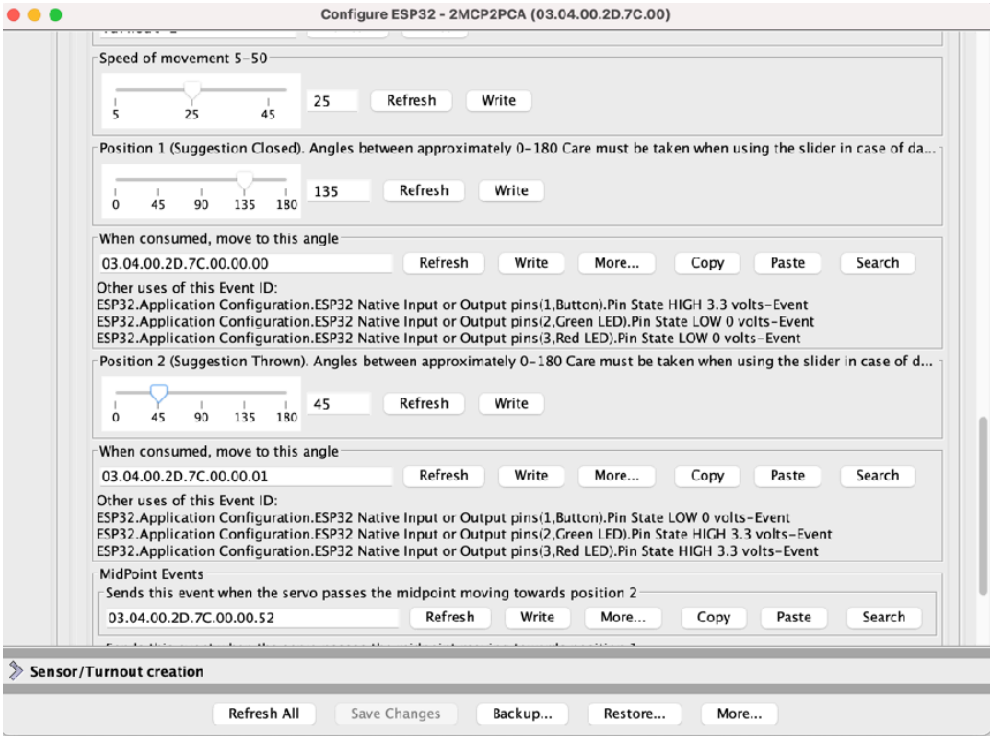
Press the Copy button for the Pin State HIGH 3.3 volts Event.

Then scroll back down to the PCA9685 modules window and Paste the event into the Position 1 section.
Don't forget to press Save Changes.



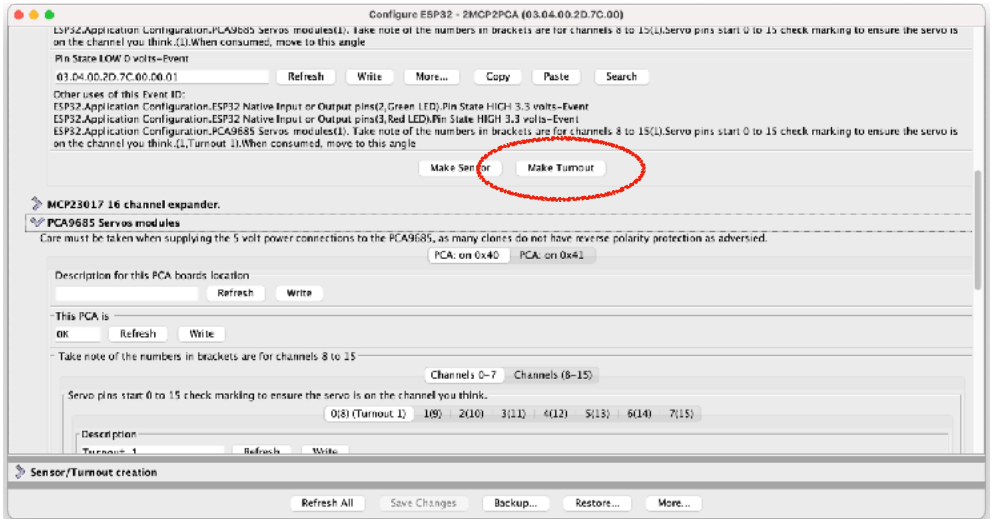
Then scroll back up and copy the Pin State LOW 0v Event,

paste it into the Position 2 section and press Save Changes.

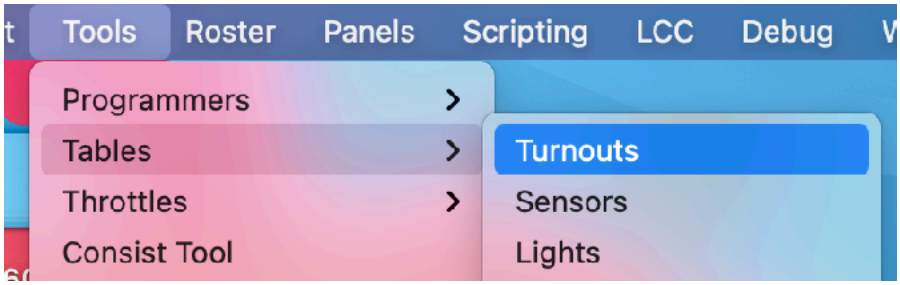


Try changing the speed and, if you wish, Positions 1 and 2.

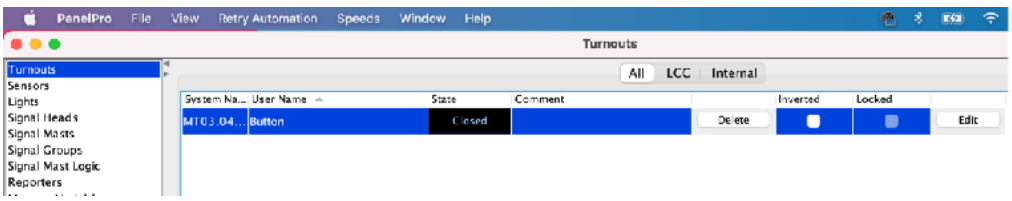
We can now make a Turnout in JMRI. Scroll up and press the Make Turnout button.



Then, in your JMRI window select Tools - Tables - Turnouts.



You'll again see that JMRI has created a Turnout so you can press the State button to toggle your servo position between Closed and Thrown.



You'll also see the State change when you press the button switch on your breadboard.

Hopefully you've now seen how easy it is to set up our MersAG_Universal sketch up in your ESP32 and how it interacts seamlessly with JMRI to control servo points and signals. Additionally, once your ESP32/MersAG_Universal has been configured using JMRI you can disconnect your computer from your ESP32 and just apply 5v to your ESP32 and still toggle your LEDs with your push-button switch.

In Part 3 we'll see how to build a Node with multiple expansion boards that can control a small layout or a section of a large and/or complex layout.

Reference

Arduino IDE
The Arduino IDE (Integrated Development Environment) is a free software application, called a sketch, used to write, compile, and upload code to physical Arduino hardware including ESP32s.

I²C Data Bus
The I²C (Inter-Integrated Circuit) bus is a two-wire serial communication protocol widely used for short-distance intra-board communication. It allows one master controller to communicate with up to 128 peripherals (slaves) using only two shared lines: the Serial Data (SDA) line and Serial Clock (SCL) line.

LED
A light-emitting diode (LED) is an electronic component that uses a semiconductor to emit light when current flows through it via 2 pins. The longer (anode) pin must be connected to positive and the shorter (cathode) pin to negative voltage.

PanelPro
An application used primarily in the context of model railroad control systems. It allows users to cr eate and manage control panels for their virtual model railroads.
PanelPro is part of the JMRI libraries, which are designed for creating control panels to manage various functions in model railroading.
It provides tools for designing control panels, including graphical editors and layout editors, enabling users to control signals, turnouts, and other functions on their layouts.

Pull-up Switch
A "pull-up switch" typically refers to a standard electronic switch (like a button) paired with a pull-up resistor. It ensures an electrical wire or microcontroller pin is held at a steady default voltage (HIGH) until the switch is activated, dropping it to zero (LOW).

Sketch
The common name for an Arduino IDE (Integrated Development Environment).